ResumeMatch - Alignment & Feature Work

What we changed on the **experiment** branch, and why

Repository: Aryan-Arora/resumematch | Branch: experiment | 10 commits (161bb78 -> da6572a) | 2026-08-01 to 2026-08-03 | 31 files, +2,256 / -30 lines

The goal

We compared the codebase against the target architecture document (an AI job-matching platform). The finding: the app was **recruiter-only** (paste a job description, upload resumes, get a ranked shortlist of candidates), but the architecture's core vision is a **job-seeker** product (upload your resume, get ranked jobs) powered by **one shared matching engine**.

Chosen direction ("Route C"): keep the recruiter tool **and** add the seeker side, both running on the same engine - one engine, two front doors. Nothing existing was thrown away; we built the missing half and reused the recruiter engine, just queried in the opposite direction (resume -> jobs instead of job -> candidates).

At a glance

- Turned a one-sided recruiter tool into a **two-sided platform** (recruiter + job seeker).
- **10 commits**, 31 files changed, +2,256 / -30 lines.
- Delivered as small, independently useful milestones (M0-M6) plus fixes and docs.
- Verified end-to-end against a live Supabase project and real Adzuna job data.

What we built (commit by commit)

Commit	Area	What it delivered
M0 (161bb78)	Hardening	Fixed the AEDT compliance-notice bug (4 job domains printed raw codes like 'skilled_trades' in a legal notice), aligned the broken local dev port (client defaulted to 4100 while the server runs on 4000), removed dead client code, and reconstructed the previously-uncommitted Supabase schema as a versioned migration.
M1 (4e78d91)	Vector search	Added pgvector HNSW cosine indexes and a match_candidates() function, plus POST /api/candidates/search - recruiters can now rank their entire candidate history against any job. Turned pgvector from a storage type into a live search engine.
M2 (1cbb910)	Job corpus + ingestion	New job_listings table and match_jobs() function, an Adzuna API client, and an 'npm run ingest' CLI that pulls real job postings, embeds each once, and caches them - the input the seeker side matches against.
M3 (f6e314f)	Seeker backend	Seeker identity and resumes (no organization required), resume upload/parse/embed, resume -> jobs matching, a per-job why/gap explainer, and a consent flag for the future talent marketplace.
M4 (325cf05)	Seeker UI	The /seeker experience in the existing React app: upload a resume, get live jobs ranked by fit with 'why you match' and 'the gap'. Extracted a shared ScoreRing component reused by both sides.
docs (e6ea282)	Setup guide	docs/SETUP.md - an end-to-end guide: apply migrations, configure both .env files, run the servers, ingest jobs, and verify the recruiter and seeker flows.

Commit	Area	What it delivered
M5 (9345665)	Application tracker	A Kanban board (Saved -> Applied -> Interviewing -> Offer -> Rejected) so seekers can track the jobs they save from their matches.
M6 (3312c87)	Talent marketplace	Recruiters search the opted-in seeker pool, anonymized - they see fit and skills, never a name or contact - closing the two-sided loop. Reuses the vector engine and the consent flag.
c7417d2	Security fix	Enabled Row Level Security on job_listings (server-only access) and made the new-table policies safely re-runnable.
da6572a	Bug fix	Seeker routes were wrongly blocked by the recruiter org-gate (returning 'no_organization'); mounted the seeker router before the org-gated routers so seekers (who have no org) can use it.

What now works

- **Recruiter:** create a job, bulk-upload resumes, get ranked and explained candidates; search the whole candidate history; browse an anonymized talent marketplace.
- **Job seeker:** upload a resume, get live jobs ranked by fit with why/gap, save them to a Kanban tracker, and opt in to be discovered by recruiters.
- **One engine** (local ONNX embeddings + pgvector) powers both directions - no matching logic duplicated.

How it was verified

- Engine classified a live job description (domain + skills) end-to-end.
- Ingested ~200 real job postings via the Adzuna API (US, then switched to India).
- Vector search returned correctly ranked matches: an app-developer resume surfaced India Android/mobile roles at the top (~56%).
- Client builds cleanly; backend health OK; all 6 migrations applied to the live Supabase project.

Deliberately deferred (future work)

- Resume tailoring - needs an external generative LLM, which breaks the current 'no external AI' design; left as a decision.
- Job alerts / saved searches.
- Double-blind contact unlock for the marketplace (the search + consent gate are built).
- Full-text job feeds (Greenhouse/Lever) for cleaner why/gap than Adzuna's short snippets.
